



MUNSCH BRYAN
RYBAK THOMAS

PROJET INFO0402
FOYER CLÉMENT

BIBLIOTHÈQUE

SVG

Sommaire

Présentation du projet.....	3
1. Objectifs	3
2. Arborescence	3
Diagramme UML.....	4
Fonctionnalités implémentées	4
1. Transformations.....	5
2. Text	5
3. Actions.....	5
4. Undo / Redo	5
5. Crop / Resize	5
6. Exportation.....	6
7. Chargement SVG	6
Conclusion	6

Présentation du projet

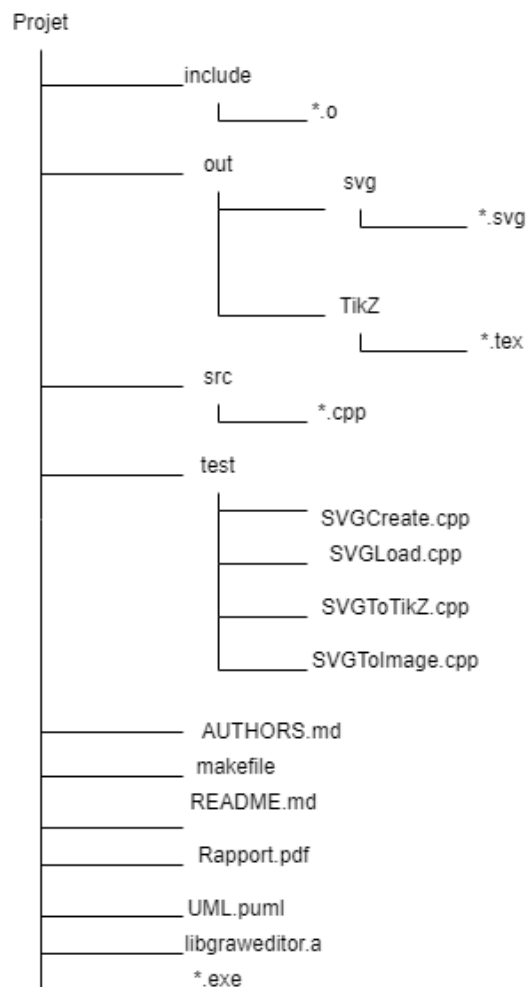
1. Objectifs

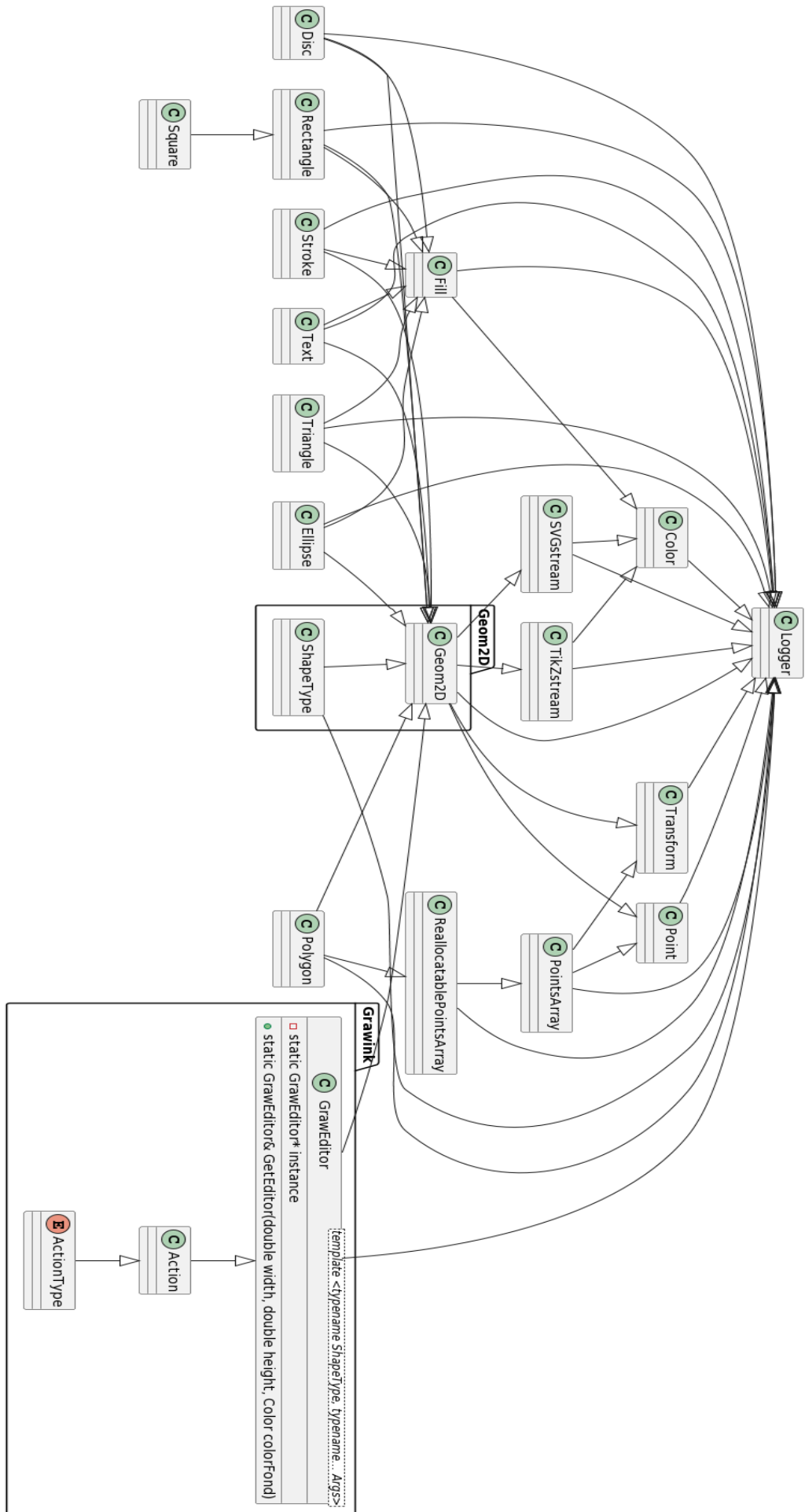
Dans le cadre du projet, nous avons pour objectif de créer une bibliothèque pour gérer des objets SVG ainsi qu'un backend tel qu'il pourrait exister pour un éditeur d'images vectorielles.

Cette bibliothèque devait nous permettre l'ajout de segments de ligne droite, de polygones réguliers (triangles, carrés, pentagones, hexagones, etc.), de rectangles, d'ellipses/cercles et d'autres formes géométriques.

Nous devons également implémenter des opérations de manipulation et de transformation d'objets tels que le redimensionnement d'une image ou encore l'annulation de la dernière action effectuée sur le canevas.

2. Arborescence





1. Transformations

Nous avons, avant toutes choses, commencé par implémenter les transformations de bases, demandées par le sujet, comme l'ajout, la suppression, la rotation, le scale, etc.

2. Text

En plus des formes proposées par le sujet, nous avons décidé de rajouter du text. Celui peut utiliser toutes les transformations citées au-dessus.

3. Actions

Pour l'utilisation des méthodes undo et redo, nous avons eu besoin d'implémenter la structure Action qui prend en paramètre une valeur provenant de l'énumération ActionType qui permet de connaître le type de l'action qui a été effectuée, comme Add, Delete, Rotate, etc.

4. Undo / Redo

Les méthodes undo et redo permettent d'annuler (undo) ou refaire (redo), **TOUTES** les actions faisables, pour tous les objets pouvant être créés (à l'exception du resize).

5. Crop / Resize

Comme le sujet le demandait, nous avons implémenter les possibilités de crop et de resize le canevas. Pour le crop, il réduit la taille du canevas et supprime les objets qui ne sont plus entièrement dedans (undo et redo fonctionne sur ces suppressions et sur la taille du canevas). Le resize redimensionne tout simplement le canevas.

6. Exportation

Une fois les fichiers .exe créés, il est possible d'exporter le résultat du programme en SVG (le format demandé par le sujet), en TikZ (format proposé dans la version avancée) et en image en .pam (avec transparence) et .ppm (sans transparence), via ces fichiers, il est possible d'obtenir un fichier .png. À savoir, le texte ainsi que les contours des formes (si changé de couleur) ne seront pas exportés.

7. Chargement SVG

Nous avons créé une méthode permettant de récupérer un fichier SVG préalablement créé pour le charger. Les formes chargées dans le SVG prennent le nom du type en minuscule et le nombre d'itération du même type auquel il se trouve. Il est possible d'utiliser toutes les fonctionnalités possibles à travers ce SVG (undo / redo, transformations, etc).

Conclusion

Dans le cadre de ce projet, nous avons atteint avec succès nos objectifs en développant une bibliothèque robuste pour la gestion d'objets SVG. Cette expérience nous a permis de mettre en pratique nos connaissances en programmation orientée objet, en manipulation d'images vectorielles. Nous avons rencontré et surmonté divers défis techniques tout au long du processus de développement, renforçant ainsi notre compréhension des concepts sous-jacents.

Pour l'avenir, nous envisageons d'améliorer davantage l'exportation en gérant d'une meilleure manière l'exportation en .pam et .ppm pour qu'ils puissent récupérer textes et contours. Il serait aussi facilement possible de gérer l'undo et redo de la méthode `resize` (cela nous a échappé).